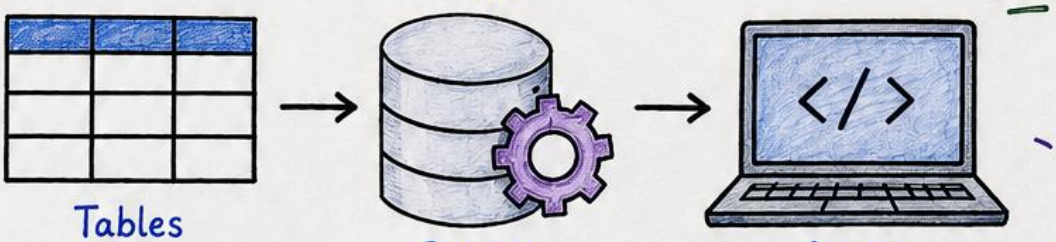
DATABASE MANAGEMENT SYSTEM

• COMPLETE NOTES •

"Data is the new oil,
but DBMS is the engine."



Organize
Store
Manage
Retrieve




```
SELECT  S.name, C.title
FROM    Student S
JOIN    Course C ON S.course_id = C.id;
```

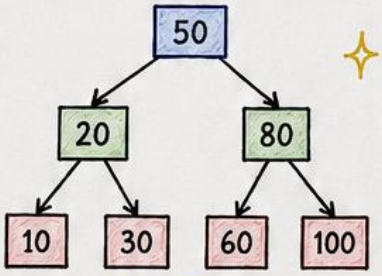
KEY TOPICS

- Theory
- Concepts
- Diagrams
- Examples
- Practice

"Well designed databases, build powerful applications."

COVERS

- ✓ Relational Model
- ✓ Relational Algebra
- ✓ SQL
- ✓ Normalization
- ✓ Transactions & Concurrency Control
- ✓ Recovery
- ✓ Indexing & File Organization
- ✓ Query Processing & Optimization



B+ Tree

Name : _____

Semester : _____

Session : 2025-26

Good database design is the foundation of great software. ♡

1. DBMS FUNDAMENTALS

Database

- A database is an organized collection of related data.

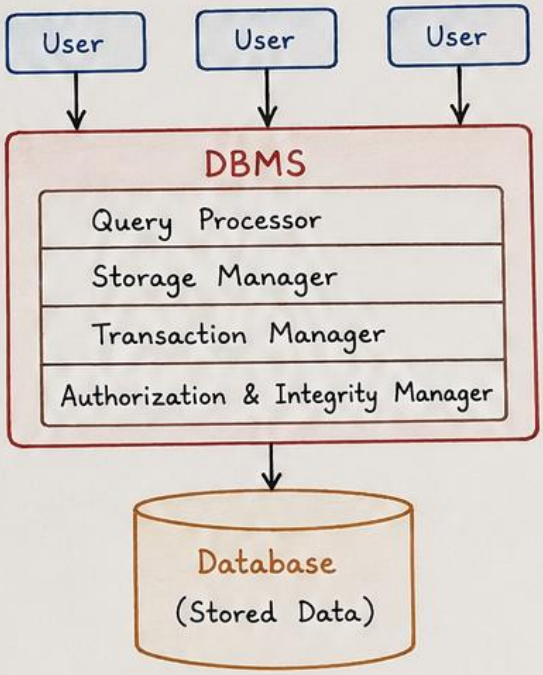
DBMS

- A DBMS is software that allows users to define, create, maintain, and control access to the database.

Purpose of DBMS

- Store large amount of data
- Provide easy access to data
- Ensure data security
- Maintain data integrity
- Reduce data redundancy
- Support data sharing

DBMS ARCHITECTURE



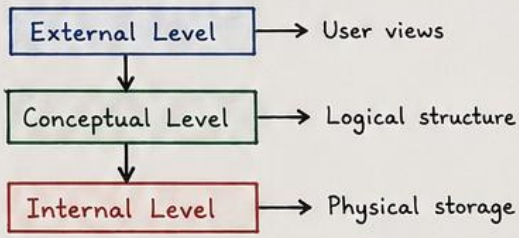
SCHEMAS AND INSTANCES

- **Schema** : The logical structure or blueprint of the database.
- **Instance** : The actual data stored in the database at a particular time.

Types of Schema

- **External Schema** : View of individual users.
- **Conceptual Schema** : Overall logical structure of the database.
- **Internal Schema** : Physical storage structure in the database.

Three Level Architecture



DATA INDEPENDENCE

- It is the ability to modify the schema at one level without affecting the schema at the next higher level.
- Types :
  1. **Physical Data Independence** : Changes in internal schema do not affect conceptual schema.
  2. **Logical Data Independence** : Changes in conceptual schema do not affect external schemas.

EXAMPLE

Schema (Blueprint)

Roll No	Name	Age	Course

Instance (Actual Data)

Roll No	Name	Age	Course
1	Rohan	20	CS
2	Ananya	21	IT

Note : DBMS helps in efficient management of data and acts as an interface between users and the database.

@codedsoul_05

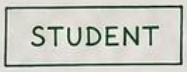
2. ER MODEL

ER Model (Entity Relationship Model) is a high level conceptual data model used to describe the structure of a database.

1. ENTITY

- An entity is a real world object that can be distinguished from other objects.
- It can be a person, place, thing or an event.
- Represented by a rectangle.

Example :



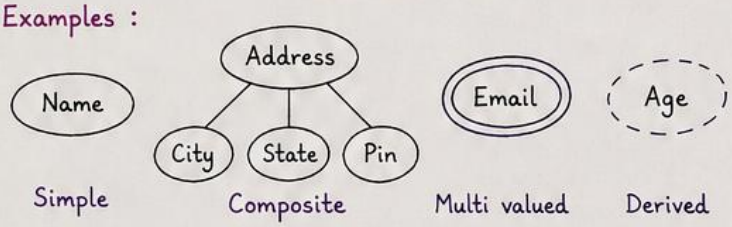
2. ATTRIBUTES

- An attribute is a property or characteristic of an entity.
- Represented by an ellipse.

Types of Attributes :

1. Simple : Cannot be divided.
2. Composite : Can be divided.
3. Single valued : Has only one value.
4. Multi valued : Can have multiple values.
5. Derived : Can be derived from other attributes.

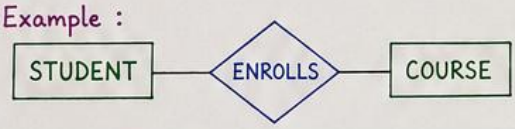
Examples :



3. RELATIONSHIP

- A relationship is an association among two or more entities.
- Represented by a diamond.

Example :

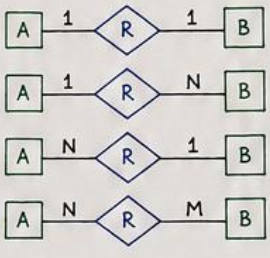


4. CARDINALITY (Mapping Cardinality)

- It defines the number of instances of one entity that can be associated with instances of another entity through a relationship.

Types :

1. One to One (1:1)
2. One to Many (1:N)
3. Many to One (N:1)
4. Many to Many (N:M)

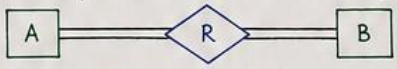


5. PARTICIPATION (Existence Dependency)

- It specifies whether the existence of an entity depends on the relationship.

Types :

1. Total Participation :
   Every instance of an entity must participate in the relationship.



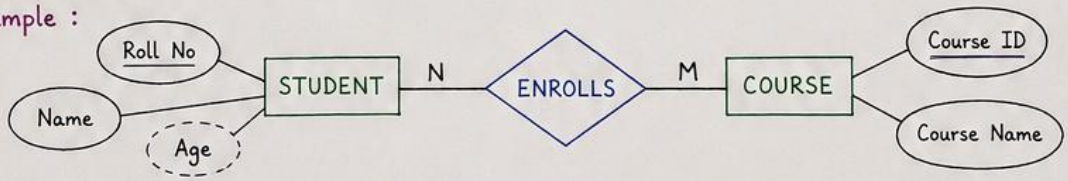
2. Partial Participation :
   Some instances of an entity may or may not participate in the relationship.



6. ER DIAGRAM

- An ER diagram is a graphical representation of entities, attributes and relationships in a database.

Example :



★ ER Model helps in designing a database before implementing it in any DBMS.

@codedsoul_05

3. RELATIONAL MODEL

The Relational Model is a logical way to represent data using tables (relations). It is the foundation of relational databases.

1. RELATION

- A relation is a table that consists of rows and columns.
- It has a unique name.

Example :

Relation Name : STUDENT

RollNo	Name	Age	Course
1	Rohan	20	CS
2	Ananya	21	IT
3	Karan	20	CS

Tuples

Attributes

2. TUPLE

- Each row in a relation is called a tuple (or record).
- It represents one instance of the relation.

Example :

(1, Rohan, 20, CS) is a tuple in STUDENT relation.

3. ATTRIBUTE

- Each column in a relation is called an attribute (or field).
- It represents a property or characteristic of the entity.

Example :

RollNo, Name, Age, Course are attributes of STUDENT relation.

4. KEYS

Keys are used to uniquely identify tuples in a relation.

① Super Key : One or more attributes that can uniquely identify a tuple.
② Candidate Key : Minimal super key.
③ Primary Key : Candidate key chosen to uniquely identify tuples.
④ Alternate Key : Candidate keys not chosen as primary key.
⑤ Foreign Key : Attribute(s) in one relation that refers to the primary key of another relation.



Example in STUDENT relation :
- Candidate Keys : {RollNo}
- Primary Key : RollNo
- Alternate Key : None

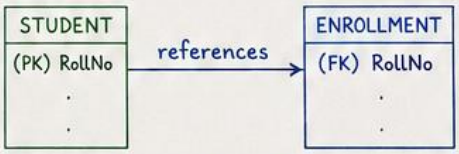
5. INTEGRITY CONSTRAINTS

These rules ensure accuracy and reliability of data in the database.

① Domain Integrity
The value in a column must come from a valid domain.
Example : Age must be a positive integer.

② Entity Integrity
Primary key values must be unique and cannot be NULL.
Example : RollNo cannot be NULL or duplicate.

③ Referential Integrity
Foreign key value must either match a primary key value in referenced relation or be NULL.



④ Key Constraints
No two tuples can have the same value for the primary key.

⑤ NOT NULL Constraint
Certain attributes cannot have NULL values.
Example : Name, RollNo cannot be NULL.

RELATION OVERVIEW

Attributes (Columns)

RollNo	Name	Age	Course
1	Rohan	20	CS
2	Ananya	21	IT
3	Karan	20	CS

→ Tuples (Rows)

In short :
Relation = Table
Tuple = Row
Attribute = Column
Key = Uniquely identifies a Row
Integrity Constraints = Rules to keep data correct

★ Relational Model is simple, powerful and widely used in all DBMS.

codedsoul_05 ♡

4. RELATIONAL ALGEBRA

Relational Algebra is a procedural query language that takes one or two relations as input and returns a relation as output.

BASIC RELATIONS (EXAMPLE)

STUDENT (S)

RollNo	Name	Dept	Age
1	Rohan	CS	20
2	Ananya	IT	21
3	Karan	CS	20
4	Neha	IT	22

COURSE (C)

CourseID	CourseName	Dept
C1	DBMS	CS
C2	OS	CS
C3	DSA	IT
C4	CN	IT

SYMBOLS USED

- σ : Selection
- π : Projection
- $\cup$: Union
- $\cap$: Intersection
- $-$: Difference
- $\times$: Cartesian Product
- $\bowtie$: Join
- $\div$: Division

1. SELECTION (σ)

Selects rows (tuples) that satisfy a condition.

Notation : $\sigma_{condition}(R)$

Example : Students from CS dept.

$\sigma_{Dept = 'CS'}(STUDENT)$

RollNo	Name	Dept	Age
1	Rohan	CS	20
3	Karan	CS	20

→ Only rows satisfying the condition are selected.

2. PROJECTION (π)

Selects columns (attributes) from a relation.

Notation : $\pi_{attribute\text{-}list}(R)$

Example : Names and Ages of students.

$\pi_{Name, Age}(STUDENT)$

Name	Age
Rohan	20
Ananya	21
Karan	20
Neha	22

→ Only specified columns are returned.

3. UNION ($\cup$)

Combines tuples from two relations.
(Both relations must be union-compatible)

Notation : $R \cup S$

Example :

Let $R(A) = \{1, 2, 3\}$ and $S(A) = \{3, 4, 5\}$

R	S	R ∪ S
A	A	A
1	3	1
2	4	3
3	5	4
		5

4. INTERSECTION ($\cap$)

Returns tuples that are common in both relations.

Notation : $R \cap S$

Example :

$R(A) = \{1, 2, 3\}$ and $S(A) = \{3, 4, 5\}$

R	S	R ∩ S
A	A	A
1	3	3
2	4	
3	5	

5. DIFFERENCE ($-$)

Returns tuples that are in first relation but not in second.

Notation : $R - S$

Example :

$R(A) = \{1, 2, 3\}$ and $S(A) = \{3, 4, 5\}$

R	S	R − S
A	A	A
1	3	1
3	5	2

6. CARTESIAN PRODUCT ($\times$)

Combines every tuple of R with every tuple of S.

Notation : $R \times S$

Example : STUDENT × COURSE

RollNo	Name	Dept	Age	CourseID	CourseName	Dept
1	Rohan	CS	20	C1	DBMS	CS
1	Rohan	CS	20	C2	OS	CS
...	...	...	...	...	...	...
4	Neha	IT	22	C4	CN	IT

7. JOIN ($\bowtie$)

Combines tuples from two relations based on a related attribute.

Types : θ-Join, Equi-Join, Natural Join

Example : Natural Join between STUDENT and COURSE on Dept.

STUDENT ⋈ COURSE

RollNo	Name	Dept	Age	CourseID	CourseName
1	Rohan	CS	20	C1	DBMS
1	Rohan	CS	20	C2	OS
2	Ananya	IT	21	C3	DSA
2	Ananya	IT	21	C4	CN
3	Karan	CS	20	C1	DBMS
...			...	...	

8. DIVISION ($\div$)

Finds tuples in R that are related to all tuples in S.
Used for 'for all' type queries.

Notation : $R \div S$

Example :

$R(Student, Course) = \{(1, C1), (1, C2), (2, C1), (2, C2), (3, C1)\}$

$S(Course) = \{C1, C2\}$

$R \div S$ → Students who took ALL courses in S.

Student
1
2

→ Student 1 and 2 have taken both C1 and C2. Student 3 has only C1.

★ Relational Algebra operations are the foundation of query processing in DBMS.

@codedsoul_05

5. SQL (Structured Query Language)

SQL is a standard language used to manage and manipulate relational databases.

1. SQL CATEGORIES

- DDL – Data Definition Language (define structure)
- DML – Data Manipulation Language (manipulate data)
- DCL – Data Control Language (access control)
- TCL – Transaction Control Language (manage transactions)

2. DDL (DATA DEFINITION LANGUAGE)

Used to define or modify database structure.

- CREATE – create object
- ALTER – modify structure
- DROP – delete object
- TRUNCATE – remove all data, keep structure

Example :

```
CREATE TABLE Student (
  RollNo INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT,
  Dept VARCHAR(20)
);
```

```
ALTER TABLE Student
ADD Email VARCHAR(50);

DROP TABLE Student;
```

3. DML (DATA MANIPULATION LANGUAGE)

Used to retrieve and manipulate data.

- SELECT – retrieve data
- INSERT – insert data
- UPDATE – modify data
- DELETE – delete data

Example :

```
INSERT INTO Student VALUES
(1, 'Rohan', 20, 'CS');
UPDATE Student SET Age = 21
WHERE RollNo = 1;
DELETE FROM Student
WHERE RollNo = 1;
```

```
SELECT * FROM Student;

SELECT Name, Age
FROM Student
WHERE Dept = 'CS';
```

4. DCL (DATA CONTROL LANGUAGE)

Used to control access to data.

- GRANT – give permissions
- REVOKE – remove permissions

Example :

```
GRANT SELECT, INSERT
ON Student TO User1;

REVOKE INSERT
ON Student FROM User1;
```



5. TCL (TRANSACTION CONTROL LANGUAGE)

Used to manage transactions.

- COMMIT – save changes
- ROLLBACK – undo changes
- SAVEPOINT – set a point within transaction

Example :

```
START TRANSACTION;
UPDATE Student SET Age = 22 WHERE RollNo = 1;
SAVEPOINT sp1;
UPDATE Student SET Age = 23 WHERE RollNo = 2;
ROLLBACK TO sp1;      -- rolls back to savepoint
COMMIT;               -- makes changes permanent
```

6. JOINS

Used to combine rows from two or more tables.

- INNER JOIN – matches in both tables
- LEFT JOIN – all from left, matched from right
- RIGHT JOIN – all from right, matched from left
- FULL JOIN – all from both
- CROSS JOIN – Cartesian product

Example : (Student / Course)

Student

RollNo	Name	Dept
1	Rohan	CS
2	Ananya	IT
3	Karan	CS

Course

CourseID	CourseName	Dept
C1	DBMS	CS
C2	OS	CS
C3	DSA	IT

```
SELECT s.Name, c.CourseName
FROM Student s INNER JOIN Course c
    ON s.Dept = c.Dept;
```

7. SUBQUERIES

A query inside another query.

Example : Students older than average age

```
SELECT Name, Age FROM Student
WHERE Age > (SELECT AVG(Age) FROM Student);
```

8. AGGREGATE FUNCTIONS

Perform calculations on a set of values.

Functions : COUNT(), SUM(), AVG(), MIN(), MAX()

Example :

```
SELECT Dept, COUNT(*) AS TotalStudents,
             AVG(Age) AS AvgAge
FROM Student
GROUP BY Dept;
```

9. GROUP BY & HAVING

- GROUP BY – groups rows with same values
- HAVING – filters groups

Example :

```
SELECT Dept, COUNT(*) AS TotalStudents
FROM Student
GROUP BY Dept
HAVING COUNT(*) > 1;
```

10. VIEWS

A view is a virtual table based on the result of a query.

Example :

```
CREATE VIEW CS_Students AS
SELECT RollNo, Name, Age FROM Student
WHERE Dept = 'CS';
-- use the view like a table
SELECT * FROM CS_Students;
```

Views simplify complex queries and hide details of data.

SUMMARY

DDL – defines structure
DML – manipulates data
DCL – controls access
TCL – manages transactions



@ codedsoul_05 ♥

6. FUNCTIONAL DEPENDENCIES (FD)

A functional dependency (FD) $X \rightarrow Y$ holds on a relation R if for any two tuples t_1 and t_2 in R, $t_1[X] = t_2[X]$ implies $t_1[Y] = t_2[Y]$.

1. FUNCTIONAL DEPENDENCY (FD)

- X and Y are sets of attributes of R.
- Written as $X \rightarrow Y$.
- Read as : X functionally determines Y.
- Y is functionally dependent on X.

Example :

Consider R (Student) : {RollNo, Name, Email, Dept}

FDs :

1) RollNo → Name, Email, Dept
2) Email → RollNo
3) Dept → Dept

RollNo uniquely identifies a student, so it determines all other attributes.

2. ATTRIBUTE CLOSURE (X^+)

The set of all attributes that X functionally determines.

How to find X^+ :

1. Start with $X^+ = X$.
2. Repeatedly apply FDs of the form $A \rightarrow B$ where $A \subseteq X^+$ to add B to X^+.
3. Stop when no more attributes can be added.

Example :

R (A, B, C, D, E) $\quad F = \{ A \rightarrow B,\ B \rightarrow C,\ AC \rightarrow D,\ D \rightarrow E \}$

Find $(A)^+$.

Step	X^+	FD Applied	New Attribute Added
0	{A}	–	–
1	{A, B}	A → B	B
2	{A, B, C}	B → C	C
3	{A, B, C, D}	AC → D	D
4	{A, B, C, D, E}	D → E	E

$(A)^+ = \{A, B, C, D, E\}$

3. ARMSTRONG'S AXIOMS (INFERENCE RULES)

These are sound and complete rules for inferring FDs.

1. **Reflexivity** : If $Y \subseteq X$, then $X \rightarrow Y$
2. **Augmentation** : If $X \rightarrow Y$, then $XZ \rightarrow YZ$
3. **Transitivity** : If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

Example :

Given : $A \rightarrow B$ (1) $\quad B \rightarrow C$ (2)

Using Transitivity (1) and (2) ⇒ $A \rightarrow C$

4. CANDIDATE KEY

A minimal super key.

- **Super Key** : A set of attributes whose closure is all attributes.
- **Candidate Key** : A super key with no proper subset being a super key.

Example :

R (A, B, C, D)

$F = \{ A \rightarrow B,\ B \rightarrow C,\ AC \rightarrow D \}$

$(A)^+ = \{A, B, C, D\}$ ⇒ A is a super key

$(AC)^+ = \{A, C, D, B\} = \{A, B, C, D\}$ ⇒ AC is a super key

No proper subset of A or AC is a super key.

Candidate Keys = { A , AC }

5. MINIMAL COVER (CANONICAL COVER)

A minimal cover of F is an equivalent set of FDs with the following properties :

1. Each FD has a single attribute on RHS.
2. No attribute in LHS is extraneous.
3. No FD is redundant.

Steps to find minimal cover for F :

1. Split RHS of each FD so that each FD has single attribute on RHS.
2. Remove extraneous attributes from LHS.
3. Remove redundant FDs.

Example :

R (A, B, C, D)

$F = \{ A \rightarrow BC,\ A \rightarrow D,\ AB \rightarrow D,\ B \rightarrow C \}$

Step 1 : Make RHS single attribute

$F_1 = \{ A \rightarrow B,\ A \rightarrow C,\ A \rightarrow D,\ AB \rightarrow D,\ B \rightarrow C \}$

Step 2 : Remove extraneous attributes from LHS

$AB \rightarrow D$ ⇒ Is B extraneous?

Check $(A)^+$ using $F_1 - \{AB \rightarrow D\} = \{A \rightarrow B, A \rightarrow C, A \rightarrow D, B \rightarrow C\}$

$(A)^+ = \{A, B, C, D\}$ ⇒ A alone determines D.

So, B is extraneous in $AB \rightarrow D$.

Replace $AB \rightarrow D$ by $A \rightarrow D$ (already present).

$F_2 = \{ A \rightarrow B,\ A \rightarrow C,\ A \rightarrow D,\ B \rightarrow C \}$

Step 3 : Remove redundant FDs

Check $B \rightarrow C$. Is it implied by others?

Using $\{ A \rightarrow B,\ A \rightarrow C,\ A \rightarrow D \}$, $(B)^+ = \{B\}$ (does not give C)

So, $B \rightarrow C$ is not redundant.

Minimal Cover, $F_c = \{ A \rightarrow B,\ A \rightarrow C,\ A \rightarrow D,\ B \rightarrow C \}$

QUICK SUMMARY

FD : $X \rightarrow Y$
Closure : X^+
Armstrong's Axioms : Reflexivity, Augmentation, Transitivity
Candidate Key : Minimal Super Key
Minimal Cover : Equivalent FD set with minimal form

★ Functional Dependencies help in analyzing relations, finding keys, and normalizing databases.

@codedsoul_05 ♥

7. NORMALIZATION

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

1. NORMAL FORMS

- 1NF → Eliminate repeating groups (atomic values)
- 2NF → Eliminate partial dependencies
- 3NF → Eliminate transitive dependencies
- BCNF → Stronger than 3NF (every determinant is a candidate key)

2. NORMAL FORMS EXPLAINED

- **1NF (First Normal Form)**

All attributes are atomic (indivisible). No repeating groups or arrays.

Example (Not in 1NF):

Student (SID, Name, Phones)

SID	Name	Phones
1	Rohan	{9876, 9123}
2	Ananya	{9988}

→

In 1NF:

SID	Name	Phone
1	Rohan	9876
1	Rohan	9123
2	Ananya	9988

- **2NF (Second Normal Form)**

Must be in 1NF and every non-prime attribute must be fully functionally dependent on the whole candidate key (no partial dependency on subset of key).

Example (Not in 2NF):

Enroll (SID, CID, SName, CName, Instructor)

SID	CID	SName	CName	Instructor
1	C1	Rohan	DBMS	Amit
1	C2	Rohan	OS	Neha
2	C1	Ananya	DBMS	Amit

Candidate Key = {SID, CID}

SName depends only on SID (partial dependency)

CName, Instructor depend only on CID (partial dependency)

Decomposition in 2NF:

Student (SID, SName)

SID	SName
1	Rohan
2	Ananya

Course (CID, CName, Instructor)

CID	CName	Instructor
C1	DBMS	Amit
C2	OS	Neha

- **3NF (Third Normal Form)**

Must be in 2NF and no transitive dependency.
Non-prime attributes should depend only on candidate key.

Example (Not in 3NF):

Emp (EmpID, EmpName, DeptID, DeptName, DeptLoc)

EmpID	EmpName	DeptID	DeptName	DeptLoc
101	Rohan	D1	IT	Mumbai
102	Neha	D1	IT	Mumbai
103	Ajay	D2	HR	Delhi

Candidate Key = EmpID

DeptName, DeptLoc depend on DeptID
(which is not a candidate key) → Transitive Dependency

Decomposition in 3NF:

Emp (EmpID, EmpName, DeptID)

EmpID	EmpName	DeptID
101	Rohan	D1
102	Neha	D1
103	Ajay	D2

Dept (DeptID, DeptName, DeptLoc)

DeptID	DeptName	DeptLoc
D1	IT	Mumbai
D2	HR	Delhi

- **BCNF (Boyce-Codd Normal Form)**

For every functional dependency X → Y,
X must be a super key.
(Stronger than 3NF)

Example (Not in BCNF):

R (RollNo, Subject, Teacher)
FDs: { RollNo → Subject, Subject → Teacher }
Candidate Key = RollNo
But Subject → Teacher and Subject is not a super key.
So, not in BCNF.

Decomposition in BCNF:

R1 (RollNo, Subject)

RollNo	Subject
1	DBMS
2	OS

R2 (Subject, Teacher)

Subject	Teacher
DBMS	Amit
OS	Neha

3. DECOMPOSITION

Breaking a relation into smaller relations to achieve higher normal forms while preserving data integrity.

4. LOSSLESS JOIN (Non-additive Decomposition)

After decomposition, joining the relations should give back the original relation without losing any data.

- Test (for R → R1, R2):
  If (R1 ∩ R2) → R1 or (R1 ∩ R2) → R2 holds in F^+,
  then the decomposition is lossless.

5. DEPENDENCY PRESERVATION

All functional dependencies in F should be enforceable using constraints on the decomposed relations without needing to join them.

- Test: The union of FDs from decomposed relations should imply F (i.e., $F^{++} = F^+$)

5. NORMAL FORMS SUMMARY

Normal Form	Conditions	What it removes	Key Point
1NF	All attributes are atomic	Repeating groups	Each cell has single value
2NF	1NF + No partial dependency (Every non-prime attribute depends on whole key)	Partial dependency	For tables with composite keys
3NF	2NF + No transitive dependency (Non-prime attributes depend only on keys)	Transitive dependency	Improves data integrity
BCNF	For every X → Y, X is a super key	More anomalies	Stricter than 3NF

QUICK RECAP

1NF → Atomic values
2NF → No partial dependency
3NF → No transitive dependency
BCNF → Every determinant is a candidate key

★ Higher the normal form, fewer the redundancies and anomalies!

@codedsoul_05

8. TRANSACTIONS

codedsoul_05 ♡

A transaction is a sequence of operations performed as a single logical unit of work.
It is the basic unit of program execution that accesses and may update various data items.

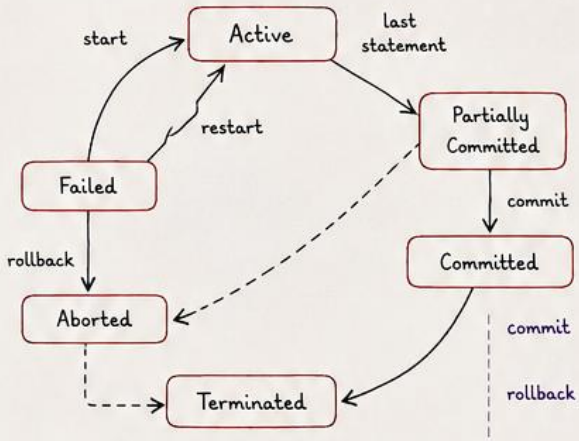
1. ACID PROPERTIES

ACID properties ensure reliable processing of transactions.

A Atomicity	All operations in a transaction are treated as a single unit. Either all succeed or none.
C Consistency	A transaction brings the database from one consistent state to another consistent state.
I Isolation	Concurrent transactions do not interfere with each other. Each transaction is isolated.
D Durability	Once a transaction is committed, its changes are permanent even if system crashes.

2. TRANSACTION STATES

A transaction goes through the following states during its lifecycle.



commit → successful completion

rollback → undo changes and go to aborted state

3. SCHEDULES

A schedule is an order of execution of operations of one or more transactions that preserves the order of operations within each transaction.

Example :

Time	T_1	T_2
1	read (A)	
2		read (B)
3	write (A)	
4		write (B)

- Here, two transactions T_1 and T_2 are interleaved.
- A schedule can be concurrent or serial.

- Serial Schedule : Transactions are executed one after another (no interleaving).
- Non-serial Schedule : Operations of transactions are interleaved.

4. SERIALIZABILITY

A schedule is correct if its result is the same as some serial schedule.

Types of Serializability :

(A) Conflict Serializability

Two operations conflict if :
1. They belong to different transactions.
2. They access the same data item.
3. At least one is write.

If a non-serial schedule can be transformed into a serial schedule by swapping non-conflicting operations, it is conflict serializable.

Precedence (Serialization) Graph



If graph is acyclic ⇒ conflict serializable

If graph has a cycle ⇒ not conflict serializable

Example :

Schedule S :
1. $r_1(X)$
2. $w_1(X)$
3. $r_2(X)$
4. $w_2(X)$

Conflicts :
- $w_1(X)$ before $r_2(X)$
- $w_1(X)$ before $w_2(X)$

Graph : $T_1 \rightarrow T_2$ (no cycle)

⇒ Conflict Serializable

(B) View Serializability

Two schedules are view equivalent if :
1. They read the same initial values.
2. They read the same values written by the same transactions.
3. They have the same final write on each item.

Every conflict serializable schedule is view serializable, but not vice versa.

Example :

Schedule S_1
1. $r_1(X)$
2. $w_1(X)$
3. $r_2(X)$
4. $w_2(X)$

Schedule S_2
1. $r_1(X)$
2. $r_2(X)$
3. $w_1(X)$
4. $w_2(X)$

S_1 and S_2 are view equivalent but not conflict equivalent.

QUICK SUMMARY

- ACID ensures reliable transaction processing.
- Transactions move through different states from active to terminated.
- Schedules define the order of operations.
- Serializability ensures correctness of concurrent execution.



9. CONCURRENCY CONTROL

@codedsoul_05 ♡

Concurrency Control is the mechanism that allows multiple transactions to execute concurrently while ensuring consistency of the database.

1. LOCKS

A lock is a mechanism to control access to data items.

- Shared Lock (S-lock) : For read operations.
  - Many transactions can hold S-lock on the same item.
  - Compatible with other S-locks.
- Exclusive Lock (X-lock) : For write operations.
  - Only one transaction can hold X-lock on an item.
  - Not compatible with any other lock.

Compatibility Matrix

Lock	S	X
S	✓	✗
X	✗	✗

✓ = Compatible
✗ = Not Compatible

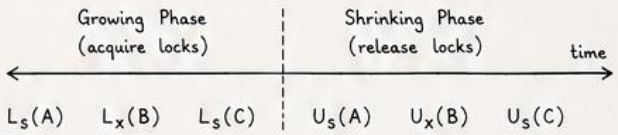
2. TWO-PHASE LOCKING (2PL)

A transaction follows two phases :

1. Growing Phase : A transaction can acquire locks but cannot release any lock.
2. Shrinking Phase : A transaction can release locks but cannot acquire any new lock.

Example (2PL)

T_i follows 2PL if its lock sequence is like below :



$L_S(X)$: acquire S-lock on X
$L_X(X)$: acquire X-lock on X
$U_S(X)$: release S-lock on X
$U_X(X)$: release X-lock on X

3. TIMESTAMP ORDERING (TO)

Each transaction is assigned a unique timestamp when it enters the system.

Rules for item X :

1. $read_i(X)$ is allowed if $TS(T_i) \geq W\text{-timestamp}(X)$.
   Then, $R\text{-timestamp}(X) = \max(R\text{-timestamp}(X), TS(T_i))$
2. $write_i(X)$ is allowed if $TS(T_i) \geq R\text{-timestamp}(X)$ and $TS(T_i) \geq W\text{-timestamp}(X)$.
   Then, $W\text{-timestamp}(X) = TS(T_i)$
3. Otherwise, the transaction T_i is aborted (rolled back).

TO ensures conflict-serializable schedules in the order of timestamps.

Example

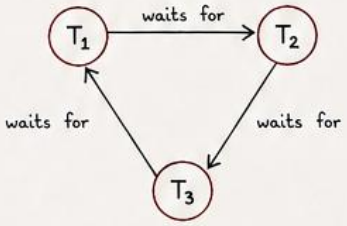
Time	T_i	Operation	R-ts(X)	W-ts(X)	Status
1	$T_1(10)$	read(X)	10	0	OK
2	$T_2(20)$	read(X)	20	0	OK
3	$T_1(10)$	write(X)	20	10	Abort (since 10 < 20 = R-ts)
4	$T_2(20)$	write(X)	20	20	OK

(Initial R-ts(X) = 0, W-ts(X) = 0)

4. DEADLOCK

A deadlock occurs when a set of transactions are each waiting for a lock held by another transaction in the set.

Example (Wait-For Graph)



Cycle in the graph ⇒ Deadlock

Deadlock Handling Techniques

1. Prevention – ensure that deadlock condition never occurs.
2. Avoidance – allow system to enter unsafe state cautiously.
3. Detection & Recovery – allow deadlock, then detect and recover.

5. MULTIPLE-GRANULARITY LOCKING (MGL)

Instead of locking individual items, lock it at multiple levels in the hierarchy:

Lock Modes

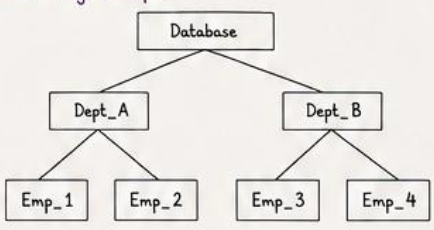
- IS : Intention Shared (will acquire S-locks on some children)
- IX : Intention Exclusive (will acquire X-locks on some children)
- S : Shared (read the node)
- X : Exclusive (write the node)

Lock Compatibility Table

	IS	IX	S	X
IS	✓	✓	✓	✗
IX	✓	✓	✗	✗
S	✓	✗	✓	✗
X	✗	✗	✗	✗

✓ = Compatible
✗ = Not Compatible

Hierarchy Example



To lock Emp_1 in X mode, T must hold IX on Dept_A and IX on Database.

QUICK SUMMARY

- Locks control access to data items.
- 2PL ensures conflict-serializable schedules.
- Timestamp Ordering uses timestamps to maintain serializability.
- Deadlock occurs due to circular wait; handle using prevention, avoidance, or detection & recovery.
- Multiple-granularity locking improves concurrency by locking at different levels.



10. RECOVERY

codedsoul_05 ♡

Recovery is the process of restoring the database to a consistent state after a failure (system crash, power failure, etc.).

1 LOG-BASED RECOVERY

- All updates are first recorded in a log file before being written to the database.
- The log contains enough information to UNDO or REDO transactions after a crash.
- Ensures Atomicity and Durability.

Log Record Format

LSN	TransID	Type	PageID	Before Image	After Image

Types: <Start>, <Update>, <Commit>, <Abort>

Example Log

LSN	TransID	Type	PageID	Before	After
10	T1	<Start>	–	–	–
20	T1	<Update>	P1	50	70
30	T2	<Start>	–	–	–
40	T2	<Update>	P2	30	90
50	T1	<Commit>	–	–	–

After crash, we use this log to UNDO or REDO.

2 CHECKPOINTS

- To avoid scanning the entire log after a crash, checkpoints are used.
- A checkpoint records the list of active transactions at that time.
- Recovery starts from the last checkpoint, reducing recovery time.

Checkpoint Record

<Checkpoint { T1, T3, T5 }> → List of active transactions at checkpoint

Log with Checkpoints (Example)

LSN	Record
100	<Start T1>
120	<Update T1, P1, 50, 70>
150	<Start T2>
170	<Checkpoint {T1, T2}>
200	<Update T2, P2, 30, 60>
230	<Commit T1>
250	<Checkpoint {T2}>

During recovery, we start scanning from the last checkpoint (LSN = 250).

3 UNDO (ROLLBACK)

- Used to reverse the effects of uncommitted transactions.
- Scan the log backwards from the end.
- For each <Update> of uncommitted txn, restore the Before Image.
- Write a <Abort> record after rollback.



Example:

Suppose T1 is uncommitted and updated P1 from 50 to 70.

1. Read <Update T1, P1, 50, 70>
2. Restore P1 to 50 (Before Image)
3. Write <Abort T1> to log



4 REDO (ROLL-FORWARD)

- Used to reapply the changes of committed transactions that may not have reached the database.
- Scan the log forwards.
- For each <Update> of committed txn, reapply the After Image.



Example:

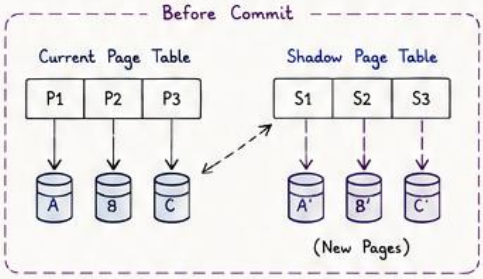
Suppose T1 is committed but its update was not on disk.

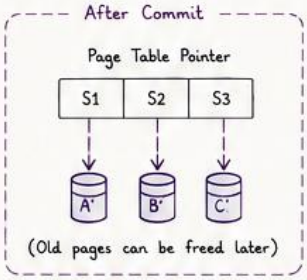
1. Read <Update T1, P1, 50, 70>
2. Write 70 (After Image) to P1
3. DB is brought up-to-date



5 SHADOW PAGING

- Maintain a shadow page table (SPT).
- All updates are first made to shadow pages.
- On commit: replace the current page table pointer with the shadow page table pointer (instant commit).
- On crash: old pages are still valid; discard shadow pages.
- No UNDO or REDO needed.





QUICK SUMMARY

Log-based Recovery	→	Uses logs to UNDO/REDO changes after failure.
Checkpoints	→	Limit the amount of log to be scanned during recovery.
UNDO	→	Rolls back the changes of uncommitted transactions.
REDO	→	Rolls forward the changes of committed transactions.
Shadow Paging	→	Uses page table switching; no UNDO/REDO required.

Goal: Restore the database to a consistent state quickly and safely after a crash.

codedsoul_05 ♡

11. INDEXING & FILE ORGANIZATION

@codedsoul_05

Indexing speeds up data retrieval. File organization decides how data is stored on disk. Good indexing + organization = less I/O, faster queries.

1. PRIMARY INDEX

- Built on the ordering field of a sequentially ordered file.
- One index entry for each block.
- Typically sparse.

Example (ordered by RollNo)

Index (on RollNo)

RollNo	Block#
101	1
111	2
121	3
131	4

Data File (sequential blocks)

Block 1	101, 102, 103, 104
Block 2	111, 112, 113, 114
Block 3	121, 122, 123, 124
Block 4	131, 132, 133, 134

2. SECONDARY INDEX

- Built on a non-ordering field.
- Multiple entries may point to the same block.
- Typically dense.

Example (index on Name)

Index (on Name)

Name	Block#
Amit	1
Amit	3
Neha	1
Rohan	2
Rohan	4
Zoya	3

Data File (blocks)

Block 1	Amit, Neha, Vijay
Block 2	Rohan, Priya
Block 3	Amit, Zoya, Karan
Block 4	Rohan, Mohit

3. DENSE vs SPARSE INDEX

Dense Index

- Has an index entry for every search-key value.
- More entries → more space.
- Faster for equality search.

Example

Index (Dense)

RollNo	Block#
101	1
102	1
103	1
104	1
105	2
106	2
...	...

Sparse Index

- Has index entry for some search-key values (usually first key of each block).
- Fewer entries → less space.
- Used with ordered files.

Example

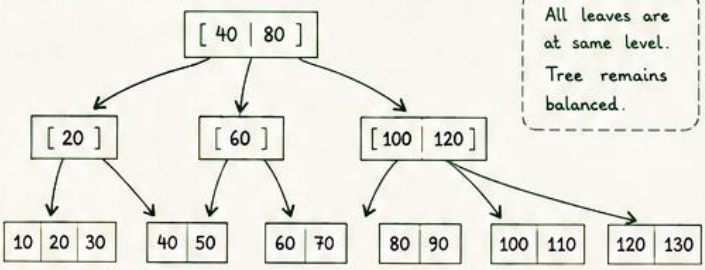
Index (Sparse)

RollNo	Block#
101	1
105	2
109	3
113	4
...	...

4. B-TREE (order m)

- Balanced multi-way search tree.
- Each node can have at most m children.
- Every node (except root) has at least [m/2] children.
- Keys in node are sorted.

Example (m = 3)

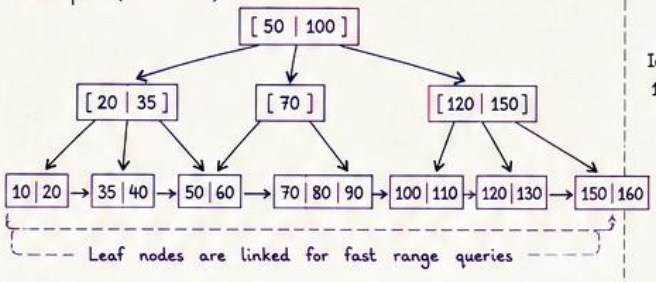


All leaves are at same level. Tree remains balanced.

5. B+ TREE

- All actual records are in leaf nodes.
- Internal nodes store only keys.
- Leaf nodes are linked (range queries efficient).
- Every node (except root) has between [m/2] and m children.

Example (order = 4)



6. HASHING

- Uses a hash function h(key) to locate the bucket.
- Very fast equality search.
- Not suitable for range queries.

Example

Hash function: h(key) = key mod 7

Buckets: 0 1 2 3 4 5 6

Insert keys: 10, 22, 31, 4, 15, 28, 17

Bucket	Keys
0	28
1	15
2	10, 31
3	17
4	4
5	22
6	—

Collision Handling Techniques

- Separate Chaining
- Open Addressing (Linear, Quadratic, Double Hashing)

QUICK SUMMARY

Topic	Key Points
Primary Index	On ordering field; one entry per block; typically sparse.
Secondary Index	On non-ordering field; multiple entries may point to same block; dense.
Dense Index	Entry for every search-key value; faster search; more space.
Sparse Index	Entry for some values (usually first of block); less space; used with ordered files.
B-Tree	Balanced tree; keys & children in all nodes; good for disk-based systems.
B+ Tree	Records in leaves; leaves linked; better for range queries; more fan-out.
Hashing	Uses hash function; best for equality search; collisions must be handled.

Key Takeaway

Indexes reduce search cost. B+ Trees handle both equality & range queries efficiently. Hashing is best for equality search with O(1) average time complexity.

12. QUERY PROCESSING & OPTIMIZATION

@codedsoul_05

Goal: Execute a SQL query efficiently and return the correct result with minimum time, CPU, I/O and memory.

1. QUERY EXECUTION OVERVIEW

Steps in query execution:

1. Parsing & Translation – Check syntax, convert SQL to relational algebra (RA).
2. Optimization – Choose the most efficient execution plan.
3. Plan Generation – Create a physical plan (operators, access paths, algorithms).
4. Execution – Run the plan and produce the result.

Example Query

```
SELECT E.name, D.name
FROM Emp E JOIN Dept D ON E.deptId = D.id
WHERE E.salary > 50000;
```

Relational Algebra (RA)

$$\pi_{E.name,\ D.name}\left(\sigma_{E.salary\ >\ 50000}\left(E \bowtie_{E.deptId\ =\ D.id} D\right)\right)$$

Projection — Selection — Join

2. JOINS

a) Types of Joins

Join Type	Notation	Meaning
Theta Join	$R \bowtie_\theta S$	θ is any condition (>, <, ≠, etc.)
Equi Join	$R \bowtie_{R.A\ =\ S.B} S$	Join with equality condition
Natural Join	$R \bowtie S$	Equi-join on common attributes, remove duplicate columns
Outer Joins	LEFT / RIGHT / FULL	Keep non-matching tuples too
Cross Join	$R \times S$	Cartesian product

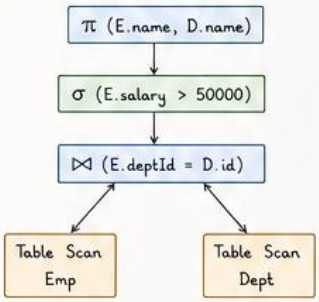
b) Join Algorithms

- Nested Loop Join – Simple, good for small inputs.
- Block Nested Loop Join – Uses blocks to reduce I/O.
- Index Nested Loop Join – Uses index on inner table.
- Sort-Merge Join – Sort both inputs, then merge.
- Hash Join – Hash one input, probe with other.

3. QUERY EXECUTION PLAN

A plan is represented as a tree of physical operators.
Leaves: access methods (table scan / index scan)
Internal nodes: relational operators (select, join, project, etc.)

Example Plan for the Query



The optimizer searches multiple plans and chooses the cheapest.

4. COST ESTIMATION

Used to estimate the cost of alternative plans.

Common Cost Parameters

- $B(T)$: number of disk blocks of table T
- $|T|$: number of tuples in T
- $V(A, T)$: number of distinct values of attribute A in T
- f_R: selection factor ($0 < f_R \leq 1$) – fraction of tuples satisfying predicate R

Example Formulas (I/O Cost)

Operation	Cost (approx.)
Full Table Scan (T)	$B(T)$
Selection $\sigma_R(T)$	$f_R \times B(T)$
Index Scan (on A)	$B(T) + \lvert T\rvert \times$ selectivity(R)
Nested Loop Join ($R \bowtie S$)	$B(R) + \lvert R\rvert \times B(S)$
Hash Join ($R \bowtie S$)	$3 \times (B(R) + B(S))$
Sort (T)	$2 \times B(T) \times \log_{M-1}(B(T))$

The optimizer uses statistics ($|T|$, $B(T)$, $V(A,T)$) to compute these costs.

5. QUERY OPTIMIZATION

a) Logical Optimization (Rule-based)

- Apply algebraic equivalences to generate equivalent RA expressions.
- Reduce the search space before physical optimization.

Common Rules

- Push Selections: $\sigma_R(R \bowtie S) \equiv (\sigma_R(R)) \bowtie S$ (if R only on R)
- Push Projections: $\pi_L(R \bowtie S) \equiv \pi_L(\pi_{L_R}(R) \bowtie \pi_{L_S}(S))$
- Join Reordering: $(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
- Use indexes, avoid unnecessary sorts.

b) Physical Optimization (Cost-based)

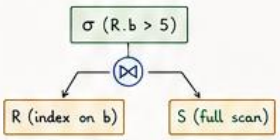
- Generate alternative physical plans.
- Estimate cost using statistics.
- Choose the plan with minimum estimated cost.

Heuristic optimization – (using rules) + Cost-based optimization – (using statistics) = Efficient plan

6. EXAMPLE: PLAN COMPARISON

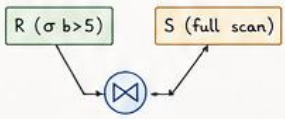
Query: SELECT * FROM R, S WHERE R.a = S.a AND R.b > 5;

Plan A: Nested Loop Join



$Cost_A = B(R_b) + |R_b| \times B(S)$
(Let $B(S) = 1000$, $|R_b| = 100$, $B(R_b) = 50$)
$Cost_A = 50 + 100 \times 1000 = 100050$

Plan B: Hash Join



$Cost_B = 3 \times (B(R_b) + B(S))$
$= 3 \times (50 + 1000) = 3150$

⇒ Plan B is better (lower cost).

7. SUMMARY

Topic	Key Points
Query Execution	Parse SQL → RA → Optimize → Generate Plan → Execute
Joins	Different join types & algorithms affect performance
Cost Estimation	Uses statistics to estimate I/O, CPU, memory costs
Query Optimization	Apply rules + cost model to find the cheapest plan
Goal	Return correct result with minimum resources

Important Takeaways

- Good statistics = good estimates = good plans.
- Smaller intermediate results = faster queries.
- Push selections & projections down.
- Choose efficient join order & algorithm.

Fun Fact

The optimizer is like Google Maps for queries – it explores many routes (plans) and picks the fastest one!

Think Smart. Optimize Early. Save Time and I/O!